

Qryx - Shopify AI Sales Assistant

Enterprise-Grade AI Chatbot Built on JNX-OS

Last Updated: 2024-12-28

Status: Planning Phase - Corrected Architecture

Type: Shopify Embedded App + Storefront Widget

Foundation: JNX-OS (Clerk Auth + Supabase + Multi-Tenant)

Executive Summary

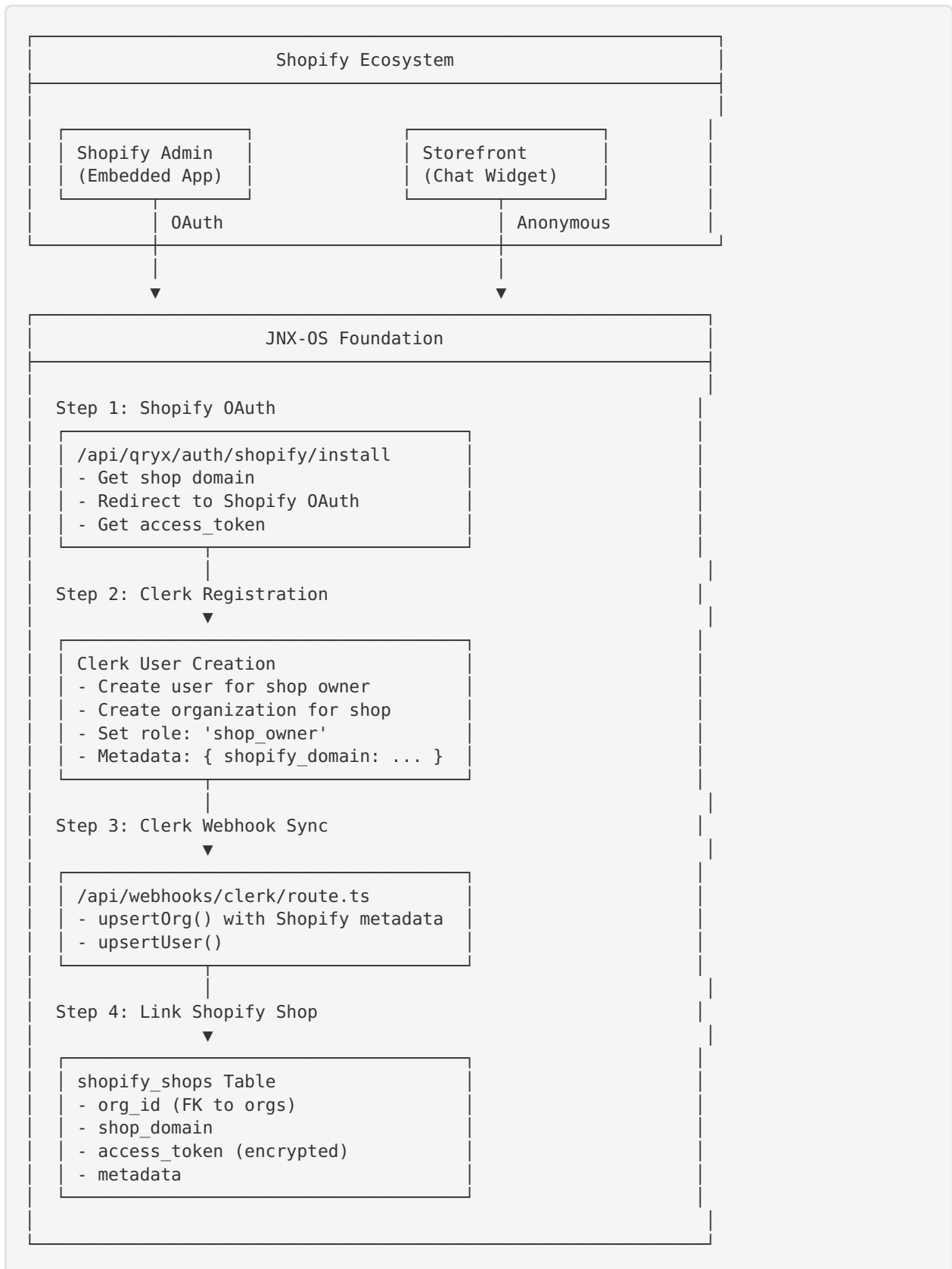
Qryx ist ein AI-powered Sales Assistant für Shopify-Shops, der **vollständig auf JNX-OS aufbaut**:

- ✓ **Built on JNX-OS** - Nutzt Clerk Auth, Supabase DB, GDPR Compliance
- ✓ **Multi-Tenant** - Jeder Shop = eine Organization in JNX-OS
- ✓ **Unified Dashboard** - Qryx-Section im bestehenden JNX-OS Dashboard
- ✓ **Shopify Integration** - OAuth für Shop-Daten, Embedded App, Storefront Widget
- ✓ **Learning Platform** - Automatisches Event-Logging via JNX SDK
- ✓ **Monetisiert** - Shopify Billing API + Subscription Management

Key Difference: Qryx ist **KEIN separates System**, sondern ein **Produkt das auf JNX-OS läuft**.

Corrected Architecture

Integration with JNX-OS



Key Architectural Decisions

1. Clerk for ALL Authentication

```
// Shop Owner wird als Clerk User angelegt
const clerkUser = await clerkClient.users.createUser({
  emailAddress: shopDetails.email,
  firstName: shopDetails.shop_owner,
  publicMetadata: {
    role: 'shop_owner',
    shopify_domain: shop_domain
  }
});

// Organization für Shop erstellen
const clerkOrg = await clerkClient.organizations.createOrganization({
  name: shopDetails.shop_name,
  publicMetadata: {
    type: 'shopify_shop',
    shopify_domain: shop_domain
  }
});

// User zur Org hinzufügen
await clerkClient.organizationMemberships.createOrganizationMembership({
  organizationId: clerkOrg.id,
  userId: clerkUser.id,
  role: 'admin'
});
```

2. Database Foreign Keys

```
-- shopify_shops MUSS org_id haben!
CREATE TABLE shopify_shops (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  org_id UUID NOT NULL REFERENCES orgs(id) ON DELETE CASCADE,
  shop_domain VARCHAR(255) NOT NULL UNIQUE,
  access_token TEXT NOT NULL, -- Encrypted
  -- ...
);

-- qryx_chat_sessions nutzt users Tabelle
CREATE TABLE qryx_chat_sessions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  shop_id UUID NOT NULL REFERENCES shopify_shops(id) ON DELETE CASCADE,
  user_id UUID REFERENCES users(id), -- Wenn Kunde eingeloggt
  -- ...
);
```

3. Unified Dashboard

```
JNX-OS Dashboard (/app)
├─ Overview (existing)
├─ Analytics (existing)
├─ Qryx (NEW)
│   ├── Chat Analytics
│   ├── Configuration
│   ├── Chat History
│   └─ Billing
└─ Settings (existing)
```

4. Reuse All Helpers

```
// Nutze bestehende helpers!
import { upsertOrg, upsertUser } from '@lib/db/helpers';
import { requireAuth, requireAdmin } from '@lib/auth/helpers';
import { redactSensitiveFields } from '@lib/privacy/redaction';
```

Corrected Database Schema

Core Tables (Existing JNX-OS)

```
-- ALREADY EXISTS in JNX-OS
orgs (
  id UUID PRIMARY KEY,
  clerk_org_id TEXT UNIQUE NOT NULL,
  name VARCHAR(255),
  metadata JSONB,
  created_at TIMESTAMPTZ,
  updated_at TIMESTAMPTZ
);

users (
  id UUID PRIMARY KEY,
  org_id UUID REFERENCES orgs(id),
  clerk_user_id TEXT UNIQUE NOT NULL,
  email VARCHAR(255),
  role VARCHAR(50),
  deleted_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ
);
```

New Tables (Qryx-Specific)

```

-- =====
-- Qryx Shopify Integration Schema
-- =====

-- Shopify Shops (Links to orgs!)
CREATE TABLE IF NOT EXISTS shopify_shops (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  org_id UUID NOT NULL REFERENCES orgs(id) ON DELETE CASCADE,

  -- Shopify Info
  shop_domain VARCHAR(255) NOT NULL UNIQUE,
  shop_name VARCHAR(255),
  shop_email VARCHAR(255),

  -- Shopify OAuth
  access_token TEXT NOT NULL, -- Encrypted via lib/security
  scope TEXT NOT NULL,

  -- Shop Details
  shop_owner VARCHAR(255),
  plan_name VARCHAR(100), -- Shopify plan (Basic, Plus, etc.)
  currency VARCHAR(10) DEFAULT 'USD',
  timezone VARCHAR(100),

  -- Installation Status
  installed_at TIMESTAMPTZ DEFAULT NOW(),
  uninstalled_at TIMESTAMPTZ,
  status VARCHAR(50) DEFAULT 'active',

  -- Qryx Subscription
  subscription_status VARCHAR(50) DEFAULT 'trial', -- trial, active, suspended
  subscription_id VARCHAR(255), -- Shopify Billing API
  trial_ends_at TIMESTAMPTZ,

  -- Metadata
  metadata JSONB DEFAULT '{}',
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW(),
  deleted_at TIMESTAMPTZ -- GDPR Soft Delete
);

CREATE INDEX idx_shops_org ON shopify_shops(org_id);
CREATE INDEX idx_shops_domain ON shopify_shops(shop_domain);
CREATE INDEX idx_shops_status ON shopify_shops(status);
CREATE INDEX idx_shops_deleted ON shopify_shops(deleted_at);

-- =====

-- Qryx Chat Sessions
CREATE TABLE IF NOT EXISTS qryx_chat_sessions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  shop_id UUID NOT NULL REFERENCES shopify_shops(id) ON DELETE CASCADE,

  -- Customer (kann anonymous oder logged-in sein)
  user_id UUID REFERENCES users(id), -- Wenn Shopify Customer eingeloggt
  customer_email VARCHAR(255),
  customer_name VARCHAR(255),
  session_token TEXT NOT NULL, -- Anonymous session ID

  -- Session Metadata
  started_at TIMESTAMPTZ DEFAULT NOW(),
  ended_at TIMESTAMPTZ,

```

```

message_count INTEGER DEFAULT 0,

-- Context
page_url TEXT,
referrer TEXT,
device_type VARCHAR(50),

-- Outcomes
has_order BOOLEAN DEFAULT FALSE,
order_id VARCHAR(255),
order_value DECIMAL(10,2),

-- Analytics
satisfaction_rating INTEGER,
tags TEXT[],

created_at TIMESTAMPTZ DEFAULT NOW(),
deleted_at TIMESTAMPTZ -- GDPR Soft Delete
);

CREATE INDEX idx_sessions_shop ON qryx_chat_sessions(shop_id);
CREATE INDEX idx_sessions_user ON qryx_chat_sessions(user_id);
CREATE INDEX idx_sessions_started ON qryx_chat_sessions(started_at DESC);
CREATE INDEX idx_sessions_deleted ON qryx_chat_sessions(deleted_at);

-- =====

-- Qryx Chat Messages
CREATE TABLE IF NOT EXISTS qryx_chat_messages (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  session_id UUID NOT NULL REFERENCES qryx_chat_sessions(id) ON DELETE CASCADE,
  shop_id UUID NOT NULL REFERENCES shopify_shops(id) ON DELETE CASCADE,

  -- Message Content
  role VARCHAR(20) NOT NULL, -- user, assistant, system
  content TEXT NOT NULL, -- PII redacted via lib/privacy/redaction

  -- AI Metadata
  model VARCHAR(100),
  tokens_used INTEGER,
  response_time_ms INTEGER,

  -- Context
  intent VARCHAR(100),
  confidence_score DECIMAL(3,2),

  -- Actions
  products_shown JSONB DEFAULT '[]',
  action_taken VARCHAR(100),

  -- Feedback
  feedback VARCHAR(20), -- positive, negative, neutral

  created_at TIMESTAMPTZ DEFAULT NOW(),
  deleted_at TIMESTAMPTZ -- GDPR Soft Delete
);

CREATE INDEX idx_messages_session ON qryx_chat_messages(session_id);
CREATE INDEX idx_messages_shop ON qryx_chat_messages(shop_id);
CREATE INDEX idx_messages_created ON qryx_chat_messages(created_at DESC);
CREATE INDEX idx_messages_deleted ON qryx_chat_messages(deleted_at);

-- =====

```

```
-- Qryx Configuration (per shop/org)
CREATE TABLE IF NOT EXISTS qryx_config (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  shop_id UUID NOT NULL UNIQUE REFERENCES shopify_shops(id) ON DELETE CASCADE,

  -- Chatbot Behavior
  bot_name VARCHAR(100) DEFAULT 'Qryx',
  bot_greeting TEXT DEFAULT 'Hi! How can I help you today?',
  bot_personality TEXT DEFAULT 'friendly',
  response_style VARCHAR(50) DEFAULT 'balanced',

  -- Features
  product_recommendations_enabled BOOLEAN DEFAULT TRUE,
  order_tracking_enabled BOOLEAN DEFAULT TRUE,

  -- Prompts
  system_prompt TEXT,
  product_context_prompt TEXT,

  -- UI Customization
  theme_color VARCHAR(7) DEFAULT '#0ea5e9',
  position VARCHAR(20) DEFAULT 'bottom-right',
  widget_enabled BOOLEAN DEFAULT TRUE,

  -- Limits
  monthly_message_limit INTEGER DEFAULT 1000,

  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_config_shop ON qryx_config(shop_id);
```

Key Schema Changes from Original

✗ OLD (Wrong):

```
CREATE TABLE shopify_shops (
  id UUID PRIMARY KEY,
  shop_domain VARCHAR(255),
  -- NO org_id! NO connection to Clerk!
);
```

✓ NEW (Correct):

```
CREATE TABLE shopify_shops (
  id UUID PRIMARY KEY,
  org_id UUID NOT NULL REFERENCES orgs(id), -- FK to JNX-OS!
  shop_domain VARCHAR(255),
  deleted_at TIMESTAMPTZ -- GDPR Soft Delete
);
```


Corrected Authentication Flow

Installation & Onboarding

Step 1: Shopify OAuth

User clicks "Install" in Shopify App Store



GET /api/qryx/auth/shopify/install?shop=myshop.myshopify.com



Redirect to Shopify OAuth



Shopify redirects to callback



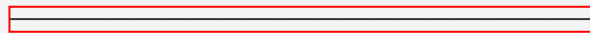
POST /api/qryx/auth/shopify/callback



Exchange code for access_token



Fetch shop details (email, owner, plan)



Step 2: Create Clerk User

Check if user exists (by shop email)



If NOT exists:



```
clerkClient.users.createUser({
  emailAddress: shopDetails.email,
  firstName: shopDetails.shop_owner,
  publicMetadata: {
    role: 'shop_owner',
    shopify_domain: shop.shop_domain
  }
})
```



Send invitation email

If EXISTS:



Get existing Clerk user



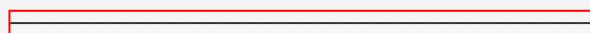
Step 3: Create Clerk Organization

```
clerkClient.organizations.createOrganization({
  name: shopDetails.shop_name,
  publicMetadata: {
    type: 'shopify_shop',
    shopify_domain: shop.shop_domain,
    plan: shopDetails.plan_name
  }
})
```



Add user as admin:

```
clerkClient.organizationMemberships.createOrganizationMembership({
  organizationId: clerkOrg.id,
  userId: clerkUser.id,
  role: 'admin'
})
```



Step 4: Clerk Webhook Sync

Clerk fires webhooks:

- user.created
- organization.created

```
- organizationMembership.created
↓
/api/webhooks/clerk/route.ts handles them
↓
Uses upsertOrg() and upsertUser()
↓
Creates records in Supabase:
- orgs (with clerk_org_id)
- users (with clerk_user_id, org_id FK)
```

Step 5: Link Shopify Shop
After webhook sync completes:

```
↓
Insert into shopify_shops:
  org_id = org.id (from Step 4)
  shop_domain = myshop.myshopify.com
  access_token = encrypt(token)
  status = 'active'
  subscription_status = 'trial'
  trial_ends_at = NOW() + 14 days
↓
```

Create default config:
INSERT INTO qryx_config (shop_id)

```
Step 6: Install Storefront Widget
Use Shopify Script Tag API:
POST /admin/api/2024-01/script_tags.json
{
  event: 'onload',
  src: 'https://jnx-os.app/qryx-widget.js?shop=myshop.myshopify.com'
}
```

Step 7: Redirect to Dashboard
Redirect shop owner to:
<https://jnx-os.app/app/qryx>

```
↓
User logs in with Clerk
↓
requireAuth() middleware
↓
Dashboard shows Qryx section
```

Implementation Code

File: /app/api/qryx/auth/shopify/callback/route.ts

```

import { NextRequest } from 'next/server';
import { clerkClient } from '@clerk/nextjs/server';
import { upsertOrg, upsertUser } from '@lib/db/helpers';
import { createSupabaseServerClient } from '@lib/supabase/server';
import crypto from 'crypto';

export async function GET(request: NextRequest) {
  const { searchParams } = new URL(request.url);
  const shop = searchParams.get('shop');
  const code = searchParams.get('code');
  const hmac = searchParams.get('hmac');

  // 1. Validate HMAC
  if (!validateHMAC(searchParams, hmac!)) {
    return Response.json({ error: 'Invalid HMAC' }, { status: 403 });
  }

  // 2. Exchange code for access token
  const tokenRes = await fetch(
    `https://${shop}/admin/oauth/access_token`,
    {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        client_id: process.env.SHOPIFY_API_KEY,
        client_secret: process.env.SHOPIFY_API_SECRET,
        code
      })
    }
  );

  const { access_token, scope } = await tokenRes.json();

  // 3. Fetch shop details
  const shopDetails = await fetchShopDetails(shop!, access_token);

  // 4. Create or get Clerk user
  let clerkUser;
  try {
    // Check if user exists
    const existingUsers = await clerkClient.users.getUserList({
      emailAddress: [shopDetails.email]
    });

    if (existingUsers.length > 0) {
      clerkUser = existingUsers[0];
    } else {
      // Create new user
      clerkUser = await clerkClient.users.createUser({
        emailAddress: shopDetails.email,
        firstName: shopDetails.shop_owner,
        publicMetadata: {
          role: 'shop_owner',
          shopify_domain: shop
        }
      });
    }
  } catch (error) {
    console.error('Clerk user creation failed:', error);
    return Response.json({ error: 'User creation failed' }, { status: 500 });
  }
}

```

```

// 5. Create Clerk organization
let clerkOrg;
try {
  clerkOrg = await clerkClient.organizations.createOrganization({
    name: shopDetails.shop_name,
    publicMetadata: {
      type: 'shopify_shop',
      shopify_domain: shop,
      plan: shopDetails.plan_name
    }
  });

  // Add user as admin
  await clerkClient.organizationMemberships.createOrganizationMembership({
    organizationId: clerkOrg.id,
    userId: clerkUser.id,
    role: 'admin'
  });
} catch (error) {
  console.error('Clerk org creation failed:', error);
  return Response.json({ error: 'Organization creation failed' }, { status: 500 });
}

// 6. Wait for webhook sync (or poll)
const org = await waitForOrgSync(clerkOrg.id);

if (!org) {
  return Response.json({ error: 'Org sync failed' }, { status: 500 });
}

// 7. Create shopify_shops record
const supabase = await createSupabaseServerClient();

const { data: shopRecord, error: shopError } = await supabase
  .from('shopify_shops')
  .insert({
    org_id: org.id,
    shop_domain: shop,
    shop_name: shopDetails.shop_name,
    shop_email: shopDetails.email,
    access_token: encryptToken(access_token),
    scope,
    shop_owner: shopDetails.shop_owner,
    plan_name: shopDetails.plan_name,
    currency: shopDetails.currency,
    timezone: shopDetails.timezone,
    status: 'active',
    subscription_status: 'trial',
    trial_ends_at: new Date(Date.now() + 14 * 24 * 60 * 60 * 1000),
    metadata: {
      shopify_id: shopDetails.id,
      country: shopDetails.country
    }
  })
  .select()
  .single();

if (shopError) {
  console.error('Shop creation failed:', shopError);
  return Response.json({ error: 'Shop creation failed' }, { status: 500 });
}

// 8. Create default config

```

```

    await supabase.from('qryx_config').insert({
      shop_id: shopRecord.id
    });

    // 9. Install storefront widget
    await installStorefrontWidget(shop!, access_token);

    // 10. Log to Learning Platform
    const { useProductLogger } = await import('@lib/jnx-products');
    const logger = useProductLogger('qryx');
    await logger.logEvent('shop_installed', {
      shop_id: shopRecord.id,
      shop_domain: shop!,
      plan: shopDetails.plan_name
    });

    // 11. Redirect to Clerk login (will redirect to /app/qryx after)
    const redirectUrl = `https://jnx-os.app/login?redirect_url=/app/qryx`;
    return Response.redirect(redirectUrl);
  }

  // Helper: Wait for webhook sync
  async function waitForOrgSync(clerkOrgId: string, maxAttempts = 10): Promise<any> {
    const supabase = await createSupabaseServerClient();

    for (let i = 0; i < maxAttempts; i++) {
      const { data } = await supabase
        .from('orgs')
        .select('*')
        .eq('clerk_org_id', clerkOrgId)
        .single();

      if (data) return data;

      // Wait 1 second before retry
      await new Promise(resolve => setTimeout(resolve, 1000));
    }

    return null;
  }

  // Helper: Encrypt token
  function encryptToken(token: string): string {
    const cipher = crypto.createCipheriv(
      'aes-256-gcm',
      Buffer.from(process.env.ENCRYPTION_KEY!, 'hex'),
      crypto.randomBytes(16)
    );
    // ... encryption logic
    return encryptedToken;
  }

```



Unified Dashboard Integration

NOT a Separate Admin Interface!

✗ WRONG Approach:

Separate Qryx Admin Dashboard at /qryx/admin

- Own auth
- Own layout
- Own navigation

✓ **CORRECT Approach:**

JNX-OS Dashboard at /app

- Add new section: /app/qryx
- Reuse existing auth (requireAuth)
- Reuse existing layout
- Add to navigation

Implementation

File: /app/app/qryx/page.tsx

```

import { requireAuth } from '@lib/auth/helpers';
import { createSupabaseServerClient } from '@lib/supabase/server';
import QryxDashboardClient from './qryx-client';

export const dynamic = 'force-dynamic';

export default async function QryxDashboardPage() {
  // Reuse existing requireAuth!
  const { clerkUser, jnxUser } = await requireAuth();

  // Check if user has a Shopify shop
  const supabase = await createSupabaseServerClient();

  const { data: shop } = await supabase
    .from('shopify_shops')
    .select('*')
    .eq('org_id', jnxUser.org_id)
    .eq('status', 'active')
    .single();

  if (!shop) {
    return (
      <div className="p-6">
        <h1>No Shopify Shop Connected</h1>
        <p>Install Qryx from the Shopify App Store first.</p>
      </div>
    );
  }

  // Fetch analytics
  const { data: analytics } = await supabase
    .from('qryx_analytics')
    .select('*')
    .eq('shop_id', shop.id)
    .order('date', { ascending: false })
    .limit(30);

  return (
    <QryxDashboardClient
      shop={shop}
      analytics={analytics || []}
      jnxUser={jnxUser}
    />
  );
}

```

File: /app/app/qryx/qryx-client.tsx


```

'use client';

import { useState } from 'react';
import { Card } from '@components/ui/card';
import { ButtonPrimary } from '@components/ui/button-primary';
import { MessageSquare, Settings, BarChart3, CreditCard } from 'lucide-react';

export default function QryxDashboardClient({ shop, analytics, jnxUser }) {
  const [activeTab, setActiveTab] = useState<'overview' | 'config' | 'history' | 'billing'>('overview');

  return (
    <div className="min-h-screen bg-slate-950">
      <div className="border-b border-slate-800 bg-slate-900/50 backdrop-blur-sm">
        <div className="max-w-7xl mx-auto px-6 py-4">
          <h1 className="text-2xl font-bold text-white">Qryx AI Assistant</h1>
          <p className="text-slate-400 mt-1">{shop.shop_name}</p>
        </div>
      </div>

      <div className="max-w-7xl mx-auto px-6 py-4">
        <div className="flex gap-2">
          <TabButton
            icon={<BarChart3 size={16} />}
            label="Overview"
            active={activeTab === 'overview'}
            onClick={() => setActiveTab('overview')}
          />
          <TabButton
            icon={<Settings size={16} />}
            label="Configuration"
            active={activeTab === 'config'}
            onClick={() => setActiveTab('config')}
          />
          <TabButton
            icon={<MessageSquare size={16} />}
            label="Chat History"
            active={activeTab === 'history'}
            onClick={() => setActiveTab('history')}
          />
          <TabButton
            icon={<CreditCard size={16} />}
            label="Billing"
            active={activeTab === 'billing'}
            onClick={() => setActiveTab('billing')}
          />
        </div>
      </div>

      <div className="max-w-7xl mx-auto px-6 py-6">
        {activeTab === 'overview' && <OverviewTab analytics={analytics} />}
        {activeTab === 'config' && <ConfigTab shop={shop} />}
        {activeTab === 'history' && <HistoryTab shop={shop} />}
        {activeTab === 'billing' && <BillingTab shop={shop} />}
      </div>
    </div>
  );
}

```

```
function TabButton({ icon, label, active, onClick }) {
  return (
    <button
      onClick={onClick}
      className={`flex items-center gap-2 px-4 py-2 rounded-lg transition ${
        active
          ? 'bg-cyan-500 text-white'
          : 'bg-slate-800/50 text-slate-400 hover:bg-slate-800'
      }`}
    >
      {icon}
      {label}
    </button>
  );
}
```

Update Main Dashboard Navigation

File: /app/app/dashboard-client.tsx

```
// Add Qryx link to navigation
<Link
  href="/app/qryx"
  className="flex items-center gap-3 px-4 py-2 rounded-lg hover:bg-slate-800/50"
>
  <MessageSquare size={20} />
  <span>Qryx Assistant</span>
</Link>
```



Extended Clerk Webhook Handler

File: /app/api/webhooks/clerk/route.ts (EXTEND existing)

```
// ADD new handler for Shopify-specific metadata
async function handleOrganizationCreated(event: any) {
  const { id, name, public_metadata } = event.data;

  // Use existing upsertOrg (idempotent!)
  const org = await upsertOrg({
    clerk_org_id: id,
    name: name,
    metadata: {
      ...public_metadata,
      // Preserve Shopify metadata
      type: public_metadata.type,
      shopify_domain: public_metadata.shopify_domain,
      plan: public_metadata.plan
    }
  });

  // If this is a Shopify org, additional setup might be needed
  if (public_metadata.type === 'shopify_shop') {
    // Log to Learning Platform
    const { useProductLogger } = await import('@lib/jnx-products');
    const logger = useProductLogger('qryx');
    await logger.logEvent('org_synced', {
      org_id: org.id,
      clerk_org_id: id,
      shopify_domain: public_metadata.shopify_domain
    });
  }

  return org;
}
```

JNX Learning Platform Integration

File: `/lib/jnx-products/qryx/config.ts` (Already created, but review)

```

import { z } from 'zod';
import { defineProduct } from '@lib/jnx-core/registry';

export default defineProduct({
  id: 'qryx',
  name: 'Qryx AI Sales Assistant',
  version: '1.0.0',

  events: {
    // Installation Events
    'shop_installed': {
      schema: z.object({
        shop_id: z.string().uuid(),
        shop_domain: z.string(),
        org_id: z.string().uuid(), // NEW: Include org_id
        plan: z.string()
      }),
      description: 'When a shop installs Qryx'
    },

    'org_synced': {
      schema: z.object({
        org_id: z.string().uuid(),
        clerk_org_id: z.string(),
        shopify_domain: z.string()
      }),
      description: 'When Clerk org is synced to Supabase'
    },

    // ... rest of events
  },

  protected: [
    'api/qryx/auth/*',
    'api/webhooks/clerk/*', // Don't touch Clerk webhooks!
    'lib/db/helpers',       // Don't modify core helpers!
  ],

  optimizable: [
    'prompts/system',
    'ui/widget-styling',
    'recommendations/logic'
  ],

  goals: {
    responseTime: { target: 2000, unit: 'ms' },
    conversionRate: { target: 0.15, unit: 'percentage' },
    userSatisfaction: { target: 0.8, unit: 'percentage' }
  }
});

```

Implementation Checklist (Corrected)

Phase 1: Database (Week 1)

- [] Run `MIGRATION_SIMPLE.sql` (if not done)
- [] Create `shopify_shops` table with `org_id` FK

- [] Create `qryx_*` tables (sessions, messages, config)
- [] Add indexes
- [] Test foreign key constraints

Phase 2: Shopify OAuth (Week 1)

- [] Create Shopify Partner Account
- [] Register Qryx app
- [] Implement `/api/qryx/auth/shopify/install`
- [] Implement `/api/qryx/auth/shopify/callback`
- [] Add HMAC validation
- [] Add token encryption

Phase 3: Clerk Integration (Week 1-2)

- [] Extend Clerk webhook handler
- [] Implement user creation flow
- [] Implement org creation flow
- [] Test webhook sync
- [] Add org metadata handling

Phase 4: Dashboard Integration (Week 2)

- [] Create `/app/app/qryx/page.tsx`
- [] Create `/app/app/qryx/qryx-client.tsx`
- [] Add Qryx to main navigation
- [] Implement analytics tab
- [] Implement config tab
- [] Implement history tab
- [] Implement billing tab

Phase 5: Chat API (Week 2-3)

- [] Create `/api/qryx/chat/route.ts`
- [] Integrate Gemini 2.0 Flash
- [] Implement session management
- [] Add PII redaction (use `lib/privacy/redaction`)
- [] Implement product recommendations

Phase 6: Storefront Widget (Week 3)

- [] Create `/public/qryx-widget.js`
- [] Create `/public/qryx-widget.css`
- [] Implement chat interface
- [] Add session tracking
- [] Add feedback buttons

Phase 7: Learning Platform (Week 3)

- [] Create qryx product config (already done)
- [] Add event logging to all actions
- [] Test event flow
- [] Verify events in database

Phase 8: Billing (Week 4)

- [] Implement Shopify Billing API
- [] Create subscription plans
- [] Add usage tracking
- [] Implement upgrade flow

Phase 9: Testing (Week 4)

- [] Test full installation flow
- [] Test chat functionality
- [] Test webhook sync
- [] Test GDPR compliance
- [] Test multi-tenant isolation



Success Criteria

✓ Correct Implementation:

1. All Shopify shops are Clerk Organizations
2. All shop owners are Clerk Users
3. `shopify_shops.org_id` references `orgs.id`
4. Qryx dashboard is at `/app/qryx` (not separate)
5. All auth uses `requireAuth()`
6. All DB operations use existing helpers
7. GDPR compliance via `lib/privacy` modules
8. Events logged to JNX Learning Platform

✓ Key Metrics:

- OAuth success rate: >95%
- Webhook sync time: <5s
- Dashboard load time: <3s
- Chat response time: <2s
- GDPR compliance: 100%



Summary of Corrections

What Changed from Original Plan?

1. Database Schema

- ✓ Added `org_id` FK to `shopify_shops`
- ✓ Added `user_id` FK to `qryx_chat_sessions`
- ✓ Added `deleted_at` for GDPR soft deletes

2. Authentication

- ✓ Shopify OAuth → Clerk Registration
- ✓ Create Clerk User + Organization
- ✓ Use Clerk Webhook for sync
- ✓ Link shop via `org_id`

3. Dashboard

- ☒ Not separate admin interface
- ☒ New section at `/app/qryx`
- ☒ Reuse `requireAuth()`
- ☒ Part of JNX-OS navigation

4. Code Reuse

- ☒ Use `upsertOrg()` and `upsertUser()`
- ☒ Use `requireAuth()` and `requireAdmin()`
- ☒ Use `redactSensitiveFields()`
- ☒ Use `createSupabaseServerClient()`

5. GDPR Compliance

- ☒ Soft deletes via `deleted_at`
- ☒ PII redaction in chat logs
- ☒ Data export via existing lib
- ☒ Data deletion via existing lib

Version: 2.0.0 (Corrected)

Last Updated: 2024-12-28

Status: Ready to Implement ☒

Foundation: JNX-OS (Clerk + Supabase)